Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

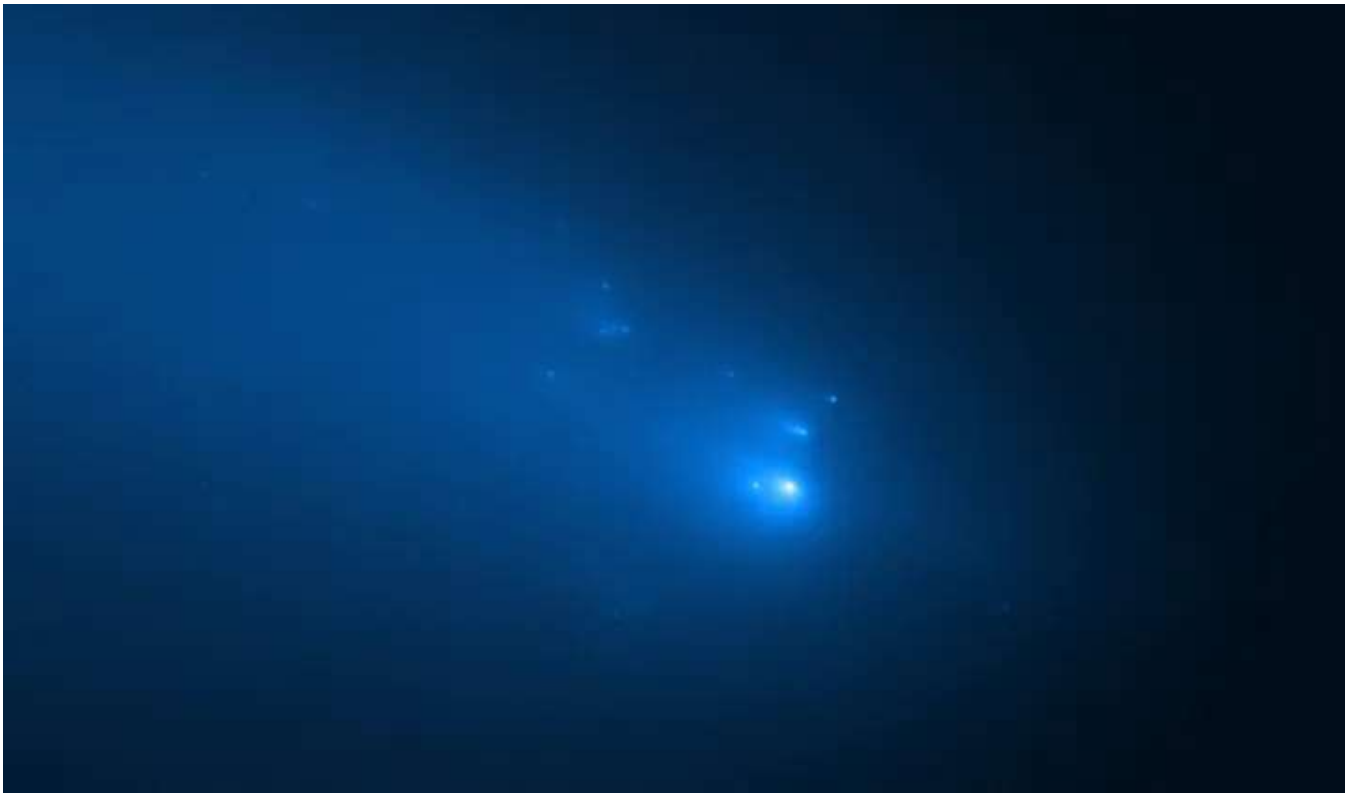
Jun 18, 2020 · 9 min read · ✨ · Listen

Save

SPACE SCIENCE WITH PYTHON

# Space Science with Python — The Solar Orbiter and comet ATLAS

Part 15 of the tutorial series describes and analyses a recent event: The flight of ESA's Solar Orbiter through the tails of comet ATLAS



Could have been a nice comet in the night sky: Comet C/2019 Y4 (ATLAS) with its remnants scattering around after a breakup of the comet's core. Image taken by NASA/ESA Hubble Space Telescope. Credit: NASA, ESA, D. Jewitt (UCLA), O. Ye (University of Maryland); License: CC BY 4.0

19

## Preface

*This is the 15th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!*

## Sun Observer meets ...

A few months ago, in February 2020, the European Space Agency (ESA) launched an ambitious mission to explore the Sun and its violent environment: The Solar Orbiter, short: *SolO*. The 1.8 ton spacecraft is equipped with 10 instruments to investigate miscellaneous properties of the Sun, like the Sun's Corona or the Solar Wind. Perihelia of around 0.3 AU will heat up the spacecraft's heat shield by a few hundred degrees Celsius. However, this engineering masterpiece will generate data that may provide us scientific insights for important Solar scientific objectives like:

- How and where is the Solar Wind originating?
- How does the Solar Dynamo work?

Cameras, magnetometers and detectors for charged particles will provide "data puzzle pieces" that will allow us to create a sophisticated picture of our nearest star.

**Solar Orbiter**

Solar Orbiter will address big questions in Solar System science to help us understand how our star creates and...

www.esa.int

## ... Sun Visitor

Two months before launch a new comet was discovered: C/2019 Y4 (ATLAS). ATLAS is a long-period comet (The Origin of Comets) with an aphelion of around 650 AU and a perihelion of around 0.25 AU. The perihelion passage was on the last day of May.

A few weeks before, observations with the Hubble Space Telescope revealed that the comet broke up into several pieces. The hope of a new *Great Comet* was gone ...

**Hubble Watches Comet ATLAS Disintegrate Into More Than Two Dozen Pieces**

Release ID: 2020-28 Comets are one of the most legendary and opulent denizens of deep space. Their long tails are so...

hubblesite.org

... *however* scientists determined that dust and ion tail of the comet pass through the trajectory of the Solar Orbiter [1]! And with its instruments, the spacecraft is able to do *bonus science* by flying through the comet's remnants.

**ESA Science & Technology - Solar Orbiter to pass through the tails of Comet ATLAS**

ESA's Solar Orbiter will cross through the tails of Comet ATLAS during the next few days. Although the recently...

sci.esa.int

*But how can scientists determine such a close encounter? What kind of parameters need to be studied to identify unplanned, great scientific opportunities in space?*

## Close Encounter

A comet has two tails:

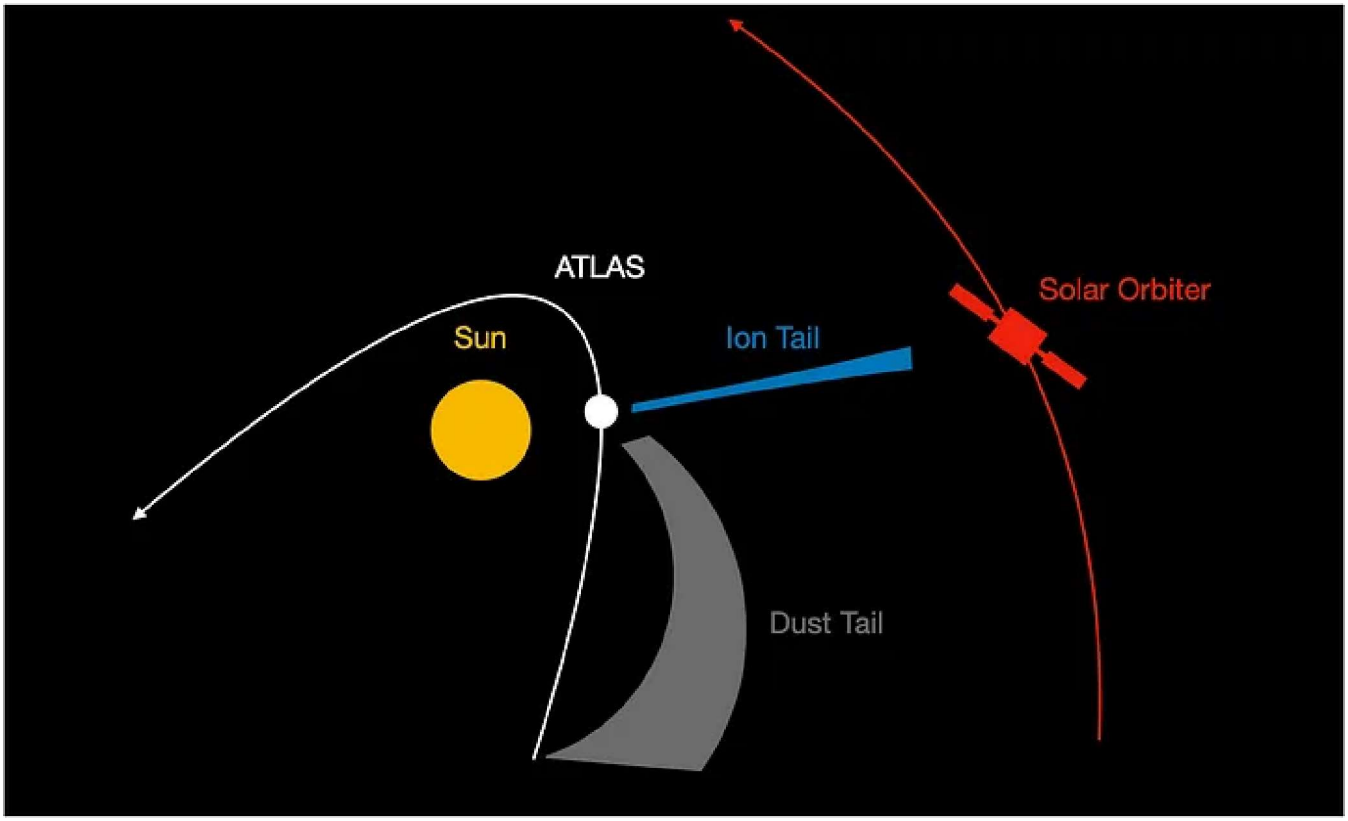
- **Dust Tail:** Due to the close distance to the Sun, the comet's core heats up and gas jets are developing. These jets transport and eject smaller dust particles from the core. Since the core has only a diameter of up to a few kilometres, an ejection speed of a few m/s is enough to leave the gravitational vicinity of the comet. The dust particles spread along the comets trajectory, but smaller ones are also affected by the Sun's radiation causing a "bending" of the tail as shown in the sketch below.

- **Ion Tail:** Charged particles, ions, leave also the comet. These particles interact strongly with the magnetic field and especially with the Solar Wind. The Solar Wind consists also of ions like Helium cores or protons and move with up to 1000 km/s. In a first approximation, these particles move radially away from the Sun. The Solar Wind wind and the radiation pressure carry the ions away from the comet: in an almost straight line.

Take a look at the sketch below. You see the Sun in the centre of our Solar System, orbited by the comet ATLAS (white) and the Solar Orbiter (red). The dust tail is

shown in grey and the ion tail points radially away from the Sun (blue). To successfully perform measurements within the ion tail, the following constraints must be fulfilled (that will be analysed in the programming part):

1. The comet and its ion tail must emerge between the Sun and the spacecraft, since the ion tail points away from the Sun.
2. The ion tail should point within a few degrees to the Solar Orbiter's direction/trajectory.
3. Timing: Let's consider an average ion tail velocity of 100 km/s [2] (it may, of course, vary significantly). The ions reach the spacecraft's trajectory after some time (depending on the distance). Ions and spacecraft must fulfil a timing constraint, otherwise, they *miss each other*.




A sketch of a comet's tails. The comet (here: ATLAS) and the spacecraft (here: Solar Orbiter) revolve around the Sun. A close encounter with the Sun causes two comet tails: A dust tail and an ion tail. The ion tail is pointing almost radially away from the Sun and should fulfil certain geometric and timing requirements to be detectable by the spacecraft. Credit: T. Albin

Let's dive into the Python code. Today, we will need again *spiceypy* and our comet database that was created in a previous session (Comets — Visitors from afar). Due to its small size, the database is also stored in my GitHub repository.

Part 1/12

The movement of comet ATLAS is determined by using the orbital elements that are stored in the database. For the Solar Orbiter, we need the corresponding spk kernel to compute the spacecraft's trajectory. ESA provides an FTP repository with all required kernels:

*ftp://spiftp.esac.esa.int/data/SPICE/SOLAR-ORBITER/kernels/spk/*

We download the kernel …

*solo_ANC_soc-orbit_20200210–20301120_L015_V1_00024_V01.bsp*

… that is also uploaded on the GitHub repository, set a relative path into the kernel meta file and load it:

Part 2/12

For the comet's orbital elements we need the G*M value of the Sun (gravitational constant times mass). We extract this information with the function *bodvcd* and assign a constant in line 3.

Part 3/12

Now, we need to extract the orbital elements from the database. The set connection (line 2) is needed to execute an SQL query in line 5 to 10 with the *pandas* function *read_sql*. All required elements are extracted that are needed later for the SPICE function *conics* that converts these information to state vectors. The name of the comet is *C/2019 Y4 (ATLAS)*.

Spatial and angular information are stored in AU and degrees, respectively. These values need to be converted to km (line 13 to 16) and radians (line 21 to 23). With *convrt* we can easily transform the km values to AU.

In line 21 to 23 a for-loop is applied that iterates through all column names that contain the sub-string *DEG* (short for *degrees*, line 21). The sub-string is replaced with *RAD* (short for *radians*, line 22) and the conversion is done with the *numpy* function *radians*. Finally, we add the G*M value of the Sun (line 26).

Part 4/12

The pandas dataframe is now converted to a list that contains the orbital elements in an order that is required for the SPICE functions:

Part 5/12

According to ESA, the tail crossing was end of May 2020. Thus, we set a date-time array starting at the 20th May 2020 (line 2) to the 6th June 2020 (line 3) in 1 hour steps (line 6 to 7).

Part 6/12

Based on the date-time array *TIME-ARRAY* we can now compute the corresponding state vectors of ATLAS and the Solar Orbiter. Two empty vectors will store the results (line 3 and 7). Line 10 to 19 and 22 to 32 perform the computation for both objects. One could easily merge both for-loops, however, I left it in separate one if one wants to add additional comet or spacecraft specific code. The for-loops iterate through the date-time array and convert it to Ephemeris time in line 13 / 25. Then, the state vector is computed in line 16 / 28 to 29 using *conics* and *spkgeo*, respectively. Finally (line 19 and 32) the positional components are appended to the lists and are later converted to *numpy* arrays in line 35 and 36.

Part 7/12

Let's check the first requirement, whether the comet is closer to the Sun than the Solar Orbiter. We compute the norm of all ATLAS and Solar Orbiter vectors and extract the minimum value (line 2 and 7). We convert the km value to AU (for better readability) and print it.

Part 8/12

Within our time window ATLAS is definitely closer to the Sun. Actually, one should analyse the distances for all time steps, but our computation covers only a few days and we already knew, thanks to the ESA article, that the comet is way closer to the Sun currently.

```
Minimum distance ATLAS - Sun in AU: 0.25281227838079623
Minimum distance Solar Orbiter - Sun in AU: 0.5208229568917663
```

Second geometric consideration: the minimum distance between the orbit of the comet and the orbit of the spacecraft. This computation can be performed from a theoretical perspective, implementing some optimization algorithm … Here, we use a quick and simple approach by comparing all position vector distances between ATLAS and the Solar Orbiter. For this purpose, we need the module *scipy*. The function *spatial.distance.cdist* requires 2 lists of vectors and computes a matrix that contains all distances between these 2 lists (line 5). We print the minimum value in line 8 to 9.

Part 9/12

The minimum distance between both orbits is around 40,000,000 km, so around 25 % of the distance between the Earth and Sun.

```
Minimum distance between ATLAS and Solar Orbiter in km: 40330530.0
```

Now we need to consider the timing! *When does ATLAS reach the point where both orbits have their minimum distance? Is it before the Solar Orbiter reaches its point?* Remember: ATLAS has to reach his point first, since the ion tail needs to develop and move towards the orbit of the Solar Orbiter. In this code part, we extract the indices of the computed distance matrix that correspond to the ATLAS and Solar Orbiter vectors that define the minimum distance (line 6 and 7). Let's print the indices for both objects.

Part 10/12

The index of ATLAS is smaller; that's great! Now let's take a look at the corresponding times. *When does ATLAS and when does the Solar Orbiter reach their orbit position where both orbits have the closest approach (line 3 to 5)?*

```
ATLAS Index of close approach: 292
Solar Orbiter Index of close approach: 503
```

Part 11/12

ATLAS reaches his point on the 1st of June 2020, the Solar Orbiter follows around 8.5 days later. Let's assume an ion tail velocity of 100 km/s. Leaving the comet's vicinity the ions travel 8,640,000 km a day. After 4.6 days the ions travelled the distance of 40 Million kilometres. 4 days later the Solar Orbiter reaches the tail. *Is it too late?* Thankfully no. The velocity of 100 km/s is a very simple approach and may differ significantly from the actual velocity distribution.

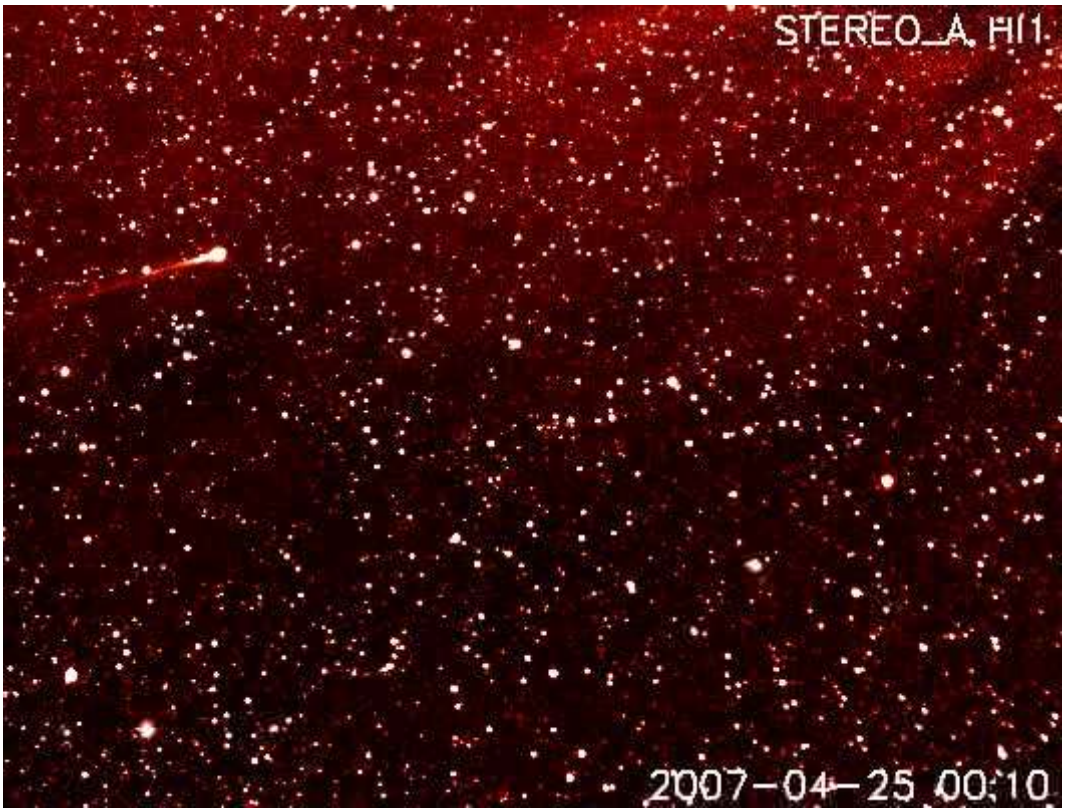
```
ATLAS closest approach date-time: 2020-06-01 04:00:00
Solar Orbiter closest approach date-time: 2020-06-09 23:00:00
```

So the distance is 40 Million kilometre (rather small in astronomical scales), and the timing appears to be feasible. The last question is: does the ion tail ***really*** point to the Solar Orbiter's direction? In our computation, we only consider the points where both orbits have their closest approach. The smallest angular distance may also be at a different time, considering the inclination of the orbits. *This may be a task for you to work on!*

*But what is the pointing of the ion tail?* Well, since the ion tail points radially away to the anti-Sun direction, we can simply use the position vector of the comet itself! We get the position vectors of ATLAS (line 7) and the Solar Orbiter (line 8) and compute their corresponding norm (line 11 and 12, using *vnorm*). The dot product is computed (line 15) to finally determine the angle between the position vector of ATLAS and the spacecraft (line 18 and 19). In line 23 to 25 we print the angular distance.

```
Minimum angular distance between a possible ion tail and the Solar
Orbiter's trajectory in degrees: 7.74
```

The minimum angular distance is 7.7 degrees. *Does it mean that we cannot detect anything with the Solar Orbiter instruments?* Well, again, it depends. The ion tail is rather narrow, but its distribution and movement are influenced severely by the radiation and Solar Wind. Consider the ion tail of comet Encke as shown below. The Sun is in the top right corner and you can easily see the tail pointing to the lower left. The tail changes its shape, width and movement all the time. These rapid changes apply also for comet ATLAS. The question is, *was the Solar Orbiter lucky enough?*




Comet Encke's ion tail is moving into the anti-Sun direction. As you can see the tail is not always a straight line and is being influenced by the Solar Wind. The Sun is at the top right corner (outside this recording), however, you can clearly see the Solar Wind due to its high density in the top right corner. Credits: NASA/SOHO

## Conclusion

With our Python and SPICE skills, we were able to re-construct a very recent event! Of course, we simplified some geometric considerations, however, we learned how

to quickly create a method to find *bonus science opportunities* for spacecraft missions that were designed for a completely different task. Go ahead, take the comet database, download more spk kernels from other spacecraft missions and maybe you find a yet unknown opportunity as a citizen scientist!

Thomas

## References

[1] Geraint H. Jones, Qasim Afghan, and Oliver Price. 2020. *Prospects for the In Situ detection of Comet C/2019 Y4 ATLAS by Solar Orbiter*. Research Notes of the AAS, Volume 4, Number 5, https://iopscience.iop.org/article/10.3847/2515-5172/ab8fa6

[2] E. Behar, H. Nilsson, P. Henri, L. Berčič, G. Nicolaou, G. Stenberg Wieser, M. Wieser, B. Tabone, M. Saillenfest, C. Goetz. 2018. *The root of a comet tail: Rosetta ion observations at comet 67P/Churyumov–Gerasimenko*. A&A 615 A21. https://www.aanda.org/articles/aa/full_html/2018/08/aa32842-18/aa32842-18.html

Python Space Science Data Science Space Science With Python

## Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play